

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ B3.2.1 Explain and apply the concept of inheritance in OOP to promote code reusability
- ▶ B3.2.2 Construct code to model polymorphism and its various forms, such as method overriding
- ▶ B3.2.3 Explain the concept of abstraction in OOP
- ▶ B3.2.4 Explain the role of composition and aggregation in class relationships
- ▶ B3.2.5 Explain commonly used design patterns in OOP

B3.2.1 Inheritance and code reusability



TOK

How does the way that we organize or classify knowledge affect what we know?

The classification of knowledge into objects, classes and inheritance in OOP affects how problems are approached and solved.



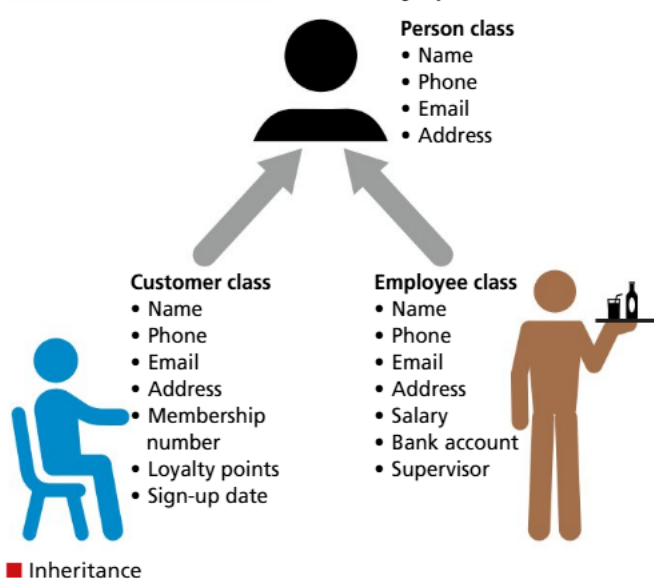
◆ **Inheritance:** where a class takes a copy of an existing class as the starting point for all its internal methods and variables. These can then be overridden and extended upon to provide additional functionality, as required.

Drawing from biology, **inheritance** is a concept that aims to ease code reuse in complex projects. It does this by allowing construction of a class that derives (inherits) existing functionality and properties from another, existing class. Once derived, you need only apply whatever custom modifications are required. The derived class is known as a “child class” or “subclass”, whereas the origin class is known as the “parent class” or “superclass”.

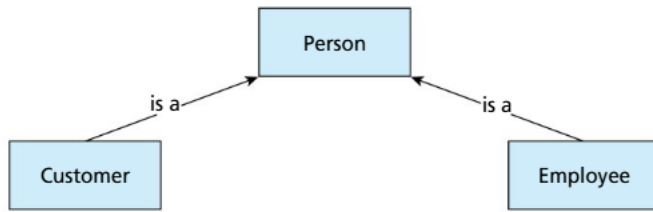
Consider writing an OOP application for a coffee shop. The business might have a loyalty scheme requiring the storing of customer details in the system. It would likely also need to store employee information to be able to pay them. While customers and employees would

have very different functions within the program, there would be some commonalities as well, such as name and contact details. Inheritance allows the programmer to put these commonalities into a parent class called **Person**, while putting the specialist functionality into the **Customer** and **Employee** classes. The common code contained within **Person** does not need to be reproduced; it is automatically available for any subclass that derives from it.

In this scenario, the **Person** class would contain the programming code responsible for any person's name, phone number, email and address. The **Customer** class inherits these basic properties and functions from **Person** and then extends upon them by adding a membership number, the points the customer has accrued and their



■ Inheritance



■ Inheritance using UML

sign-up date. The `Employee` class likewise inherits the basic properties and functions of `Person` and extends them by adding salary and bank-account details, along with information about an individual employee's supervisor.

Drawn using UML, this inheritance relationship is denoted via an "is-a" arrow pointing from the derived class to the parent class.

The following code blocks illustrate how to create these three classes in code. Note particularly that:

- in Python:
 - the invoking of the inheritance relationship occurs through the use of parenthesis in the class definition, e.g. `class Customer(Person)` creates a class `Customer`, inheriting from `Person`
 - the constructor of the superclass must still be called; this must be the first line in the subclass constructor, achieved through `super().__init__()`
- in Java:
 - the invoking of the inheritance relationship occurs through the use of the `extends` keyword in the class definition, e.g. `class Customer extends Person` creates a class `Customer`, inheriting from `Person`
 - the constructor of the superclass must still be called; this must be the first line in the subclass constructor, achieved through `super()`
 - the protected access modifier will allow the subclasses to directly access the instance variables that have been defined within the superclass.

Python

```

class Person:
    def __init__(self, name, phone, email, address):
        self.name = name
        self.phone = phone
        self.email = email
        self.address = address
    def __str__(self):
        return "Person: "+self.name

class Customer(Person): # The parenthesis indicates Customer inherits Person
    def __init__(self, name, phone, email, address, membership, points, signup):
        # Call the constructor of the parent class
        super().__init__(name, phone, email, address)
        # Define other instance variables specific to this child class
        self.membership = membership
        self.points = points
        self.signup = signup
    def __str__(self):
        return "Customer: "+self.name+" has "+str(self.points)+" points."
  
```

```
class Employee(Person): # The parenthesis indicates Employee inherits Person
    def __init__(self, name, phone, email, address, salary, bankaccount, supervisor):
        # Call the constructor of the parent class
        super().__init__(name, phone, email, address)
        # Define other instance variables specific to this child class
        self.salary = salary
        self.bankaccount = bankaccount
        self.supervisor = supervisor
    def __str__(self):
        return "Employee: "+self.name+" earns $" +str(self.salary)
```

Top tip!

When first learning how to use dates and times in Java, you will discover a multitude of different options. New approaches have evolved as the language has matured, while the old approaches had to be retained for the language to be backwards-compatible for older projects. Since Java 8, the `java.time` package is the recommended best practice approach for managing dates and times.

Java

```
import java.time.LocalDate;
class Person {
    // By using protected instead of private, these variables will be accessible
    // by classes that inherit Person. Refer to the section on access modifiers
    // for more.
    protected String name;
    protected String phone;
    protected String email;
    protected String address;
    Person(String name, String phone, String email, String address) {
        this.name = name;
        this.phone = phone;
        this.email = email;
        this.address = address;
    }
    public String toString() {
        return "Person: "+this.name;
    }
}
class Customer extends Person { // The "extends" keyword indicates inheritance
    private long membership;
    private long points;
    private LocalDate signup;
    Customer(String name, String phone, String email, String address, long
    membership, long points, LocalDate signup) {
        // Call the constructor of the parent class
```

```

        super(name, phone, email, address);
        // Initialize other instance variables
        this.membership = membership;
        this.points = points;
        this.signup = signup;
    }
    @Override
    public String toString() {
        return "Customer: "+super.name+" has "+Long.toString(this.points)+" points";
    }
}

class Employee extends Person { // The "extends" keyword indicates inheritance
    private long salary;
    private String bankaccount;
    private Person supervisor;
    Employee(String name, String phone, String email, String address, long
salary, String bankaccount, Person supervisor) {
        // Call the constructor of the parent class
        super(name, phone, email, address);
        // Initialize other instance variables
        this.salary = salary;
        this.bankaccount = bankaccount;
        this.supervisor = supervisor;
    }
    @Override
    public String toString() {
        return "Employee: "+super.name+" earns $" +Long.toString(this.salary);
    }
}

```

● Common mistake

Overusing inheritance can lead to tightly coupled code that is difficult to modify. Use inheritance sparingly – only when classes share a logical and robust “is-a” relationship. If two classes do not share enough functionality, consider alternatives such as composition.

ACTIVITY



Research skills and thinking skills: Organize a class debate around the value of inheritance. Does inheritance in object-oriented programming lead to better designed and more efficient code, or does it overly restrict and limit flexibility?

B3.2.2 Polymorphism and method overriding

◆ **Polymorphism:** meaning “many forms”, it allows objects to exhibit different behaviours based on their specific class implementation while still adhering to a shared interface or contract.

Polymorphism refers to how related objects can perform the same task or interaction in a different way. “Polymorphism” is another term that Computer Science has taken from biology; it refers to something that can take many forms (*poly* means many and *morph* means change form).

Overriding occurs when a child class creates a property or method of the same name as the parent class, thereby overriding it.

You can see an example of polymorphism occurring in the example above. The Python `__str__()` functions and the Java `toString()` functions are overridden in the child classes of `Customer` and `Employee`. Creating main code of what follows will demonstrate that it is, in fact, the `__str__()` and `toString()` functions of the child classes that execute, rather than those in the parent classes.

If `__str__()` / `toString()` was not defined in the subclass, then the version that exists in the superclass is what would be executed.

Python

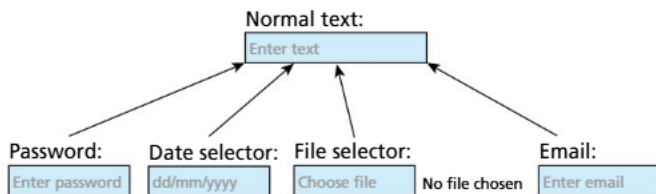
```
from datetime import datetime
if __name__ == "__main__":
    p = Person("Jordan McFly", "555 1234", "jordan@example.com", "1885 Brown Estate")
    c = Customer("Skyler Serenity", "555 2345", "skyler@example.com", "1701 Asimov Plaza", 1, 0, datetime.now())
    e = Employee("Avery Shephard", "555 3456", "avery@example.com", "1955 Lone Pine Mall", 75000, "123-456-888", p)
    print(p)
    print(c)
    print(e)
```

Java

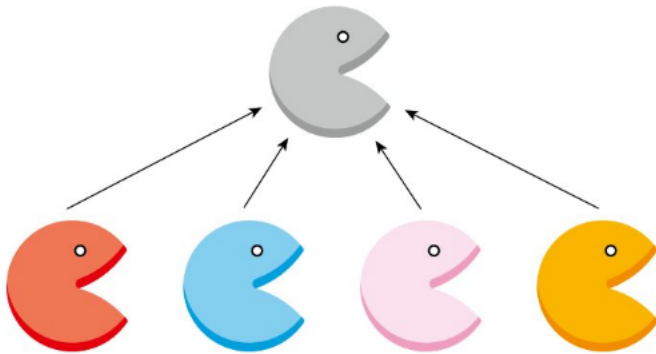
```
import java.time.LocalDate;
class Main {
    public static void main(String[] args) {
        Person p = new Person("Jordan McFly", "555 1234", "jordan@example.com", "1885 Brown Estate");
        Customer c = new Customer("Skyler Serenity", "555 2345", "skyler@example.com", "1701 Asimov Plaza", 1, 0, LocalDate.now());
        Employee e = new Employee("Avery Shephard", "555 3456", "avery@example.com", "1955 Lone Pine Mall", 75000, "123-456-888", p);
        System.out.println(p);
        System.out.println(c);
        System.out.println(e);
    }
}
```

These should render the following output, demonstrating the different versions of the `toString()` function are executing.

Person: Jordan McFly
 Customer: Skyler Serenity has 0 points
 Employee: Avery Shephard earns \$75000



■ Inheritance in graphical user interfaces



■ Inheritance in game design

■ Example uses

Two examples that illustrate the benefit of inheritance combined with polymorphic overriding are:

- Consider the variety in input boxes available for graphical user interfaces. The core functionality of a text input box can be written once, and then inheritance can be used to use that functionality as the basis for more specialized input types, such as *PasswordInput*, *DateInput*, *FileInput* or *EmailInput*, where some of the original code is overridden to provide the new, specialized experience.
- A second example is in the area of game design. A game that possesses multiple bots to play against could have one generic bot with a random move strategy. Specialized bots could then inherit this basic code and override the strategy function, or the abilities function.

■ Overriding default methods

Earlier in this section, you **overrode** the `toString()` / `__str__()` functions in Java and Python respectively. These are automatically called whenever the context calls for a string value from the object, such as being used by a `print()` method.

There are other default method names that are handy to know about so you can override their behaviour when appropriate. Some of these are:

- in Java:
 - `equals(Object obj)`: checks whether another object passed to it is “equal to” the current instance; the default implementation checks for reference equality (i.e. whether they point to the same object in memory) – overriding this method allows you to compare the contents of two objects for logical equality
 - `hashCode()`: returns an integer hash code value for the object, which is used by hash-based collections like `HashMap` and `HashSet`; when you override `equals()`, you must also override `hashCode()` to maintain the general contract for the `hashCode()` method, which states that equal objects must have equal hash codes – see Section B4.1.6 for an explanation of what this means
- in Python:
 - `__eq__(self, other)`: called when the equality operator `==` is used to compare two objects; overriding it allows for custom comparison logic

◆ Overriding:

the process of providing a different implementation of a method in a subclass, which replaces the original implementation inherited from the superclass.

- `__hash__(self)`: returns an integer hash value for the object, and is used in hashable collections such as sets and dictionaries; if you override `__eq__`, you should also override `__hash__`, ensuring that objects that are considered equal have the same hash value – see Section B4.1.6 for an explanation of what this means
- `__lt__(self, other)`, `__le__(self, other)`, `__gt__(self, other)`, `__ge__(self, other)`: used for comparison operators `<`, `<=`, `>` and `>=` respectively; they are used by the `sorted()` function and other areas where there is a concept of an ordering of objects
- `__getitem__(self, key)`, `__setitem__(self, key, value)`, `__delitem__(self, key)`: called to retrieve, set or delete an item using the indexing syntax `obj[key]`
- `__iter__(self)` and `__next__(self)`: used to make an object iterable (usable in a FOR loop, for example); `__iter__` should return an iterator object, which is typically the object itself, and `__next__` should return the next item or raise `StopIteration` to end the iteration.

Refer to the Java and Python documentation for examples of implementing each of the methods above, as relevant.

B3.2.3 Abstraction and abstract classes

One common way of making use of inheritance and polymorphism is through the use of abstract classes. Abstract classes can be thought of as generic templates without any executable code of their own. They are classes that cannot be instantiated. They are designed to be extended by other classes.

You use abstract classes when you know that all your subclasses should have certain methods or fields, but the implementation of these methods is inherently specific to each subclass, such that it doesn't make logical sense to provide a default implementation.

While it doesn't provide any functionality, the abstract class provides standardization and consistency in the implementation of subclasses; the benefit being that anyone using your class hierarchy will know that certain methods are always available, and any new derived classes need to adhere to the defined contract.

As an example, consider an application that needs objects to manage the properties of various 2D geometric shapes, such as a possible computer game. It makes sense to require that all classes that implement a 2D shape have a function that returns its surface area, and another that returns its perimeter. An abstract class of `Shape` can be defined that stipulates these requirements, as the following example demonstrates.

Notice that, because Python and Java can rely upon the contract associated with `Shape`, there is no problem creating code that executes `getPerimeter()` and `getArea()` on an array containing a mix of `Rectangle` and `Circle` objects.

Java

```

abstract class Shape{
    public abstract double getArea();
    public abstract double getPerimeter();
}
class Rectangle extends Shape {
    protected double width;
    protected double height;
    Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }
    @Override // Specify we are overriding the abstract method
    public double getArea() {
        return this.width * this.height;
    }
    @Override // Specify we are overriding the abstract method
    public double getPerimeter() {
        return 2 * (this.width + this.height);
    }
}
class Circle extends Shape {
    protected double radius;
    Circle(double radius) {
        this.radius = radius;
    }
    @Override
    public double getArea() {
        return Math.PI*this.radius*this.radius;
    }
    @Override
    public double getPerimeter() {
        return 2*Math.PI*this.radius;
    }
}
class Main {
    public static void main(String[] args) {
        Shape[] shapes = new Shape[4];
        shapes[0] = new Rectangle(10.0, 4.0);
        shapes[1] = new Rectangle(36.0, 7.0);
        shapes[2] = new Circle(42.0);
        shapes[3] = new Circle(10.0);
        for (int i=0; i<shapes.length; i++) {
            System.out.println( shapes[i].getArea() );
            System.out.println( shapes[i].getPerimeter() );
        }
    }
}

```


Python

```

from abc import ABC, abstractmethod
import math

# Shape: Abstract class in Python, inherits from ABC (Abstract Base Class)
class Shape(ABC):
    @abstractmethod
    def get_area(self):
        pass
    @abstractmethod
    def get_perimeter(self):
        pass

# Rectangle class, inherits from Shape
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def get_area(self):
        return self.width * self.height
    def get_perimeter(self):
        return 2 * (self.width + self.height)

# Circle class, inherits from Shape
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def get_area(self):
        return math.pi * self.radius ** 2
    def get_perimeter(self):
        return 2 * math.pi * self.radius

# Main section
if __name__ == "__main__":
    shapes = [
        Rectangle(10.0, 4.0),
        Rectangle(36.0, 7.0),
        Circle(42.0),
        Circle(10.0)
    ]
    for i in range(len(shapes)):
        print(shapes[i].get_area())
        print(shapes[i].get_perimeter())

```



Common mistake

Confusing the role of abstract classes

It is common to misunderstand the purpose of abstraction, and to use it where a simple base class with inheritance might suffice. Use abstract classes when you have a base class that should not be instantiated itself, but has common code to share with the subclasses that are instantiated. Make sure any subclass implements all the abstract methods from the abstract superclass.

B3.2.4 Composition and aggregation

Because a class is analogous to a data type, and an object is analogous to a variable, you can quickly end up with programs where an object contains many other objects of different classes as variables within it, and this process can repeat itself into multiple layers of depth. When this occurs, these different objects are said to be related to each other and dependent on each other.

There are different ways of defining these dependent relationships. Inheritance, for example, is a type of dependent relationship where the child class requires the parent class to exist as its original source of properties and methods.

There are two other means of defining dependent relationships that we will look at now: composition and aggregation.

■ Composition

Composition is where one object is composed of one or more objects, and the composed objects cannot exist without the containing object. This creates a strong “one is part of the other” relationship, where the lifetime of the composed objects is managed by the container. This can even be referred to as a “death relationship”, meaning if the whole is destroyed, its parts are destroyed as well.

Some examples of composition relationships include:

- **Car and Engine:** A Car object is composed of an Engine object. The engine is an integral part of the car. Outside of the car, it does not serve the purpose it was designed for. If the car is destroyed, the engine is as well.
- **House and Room:** A House object is composed of multiple Room objects. Rooms are part of a house; if the house is demolished, the rooms also cease to exist as functional units.
- **Computer and Components:** A Computer object is composed of CPU, Motherboard, Memory, and so on. These components are parts of a computer and do not function independently if separated.
- **Human and Organs:** A Human object is composed of a Heart, a Brain and Lungs. These organs are essential parts of a human body, and they do not function outside the body.

■ Aggregation

Aggregation is where one object contains one or more objects, but the contained objects can exist independently without the “container”. Should the containing object cease to exist, the previously contained objects may still exist in their own right. In this way, the lifecycle of the contained objects is not managed by the containing object.

Some examples of aggregation relationships include:

- **University and Student:** A University contains many Student objects. If the university was to close (or once a student completes their studies), the student continues to exist even when no longer attached to the university.
- **Library and Books:** A Library contains many Book objects. The library has books, but the books can exist outside of the library as well.
- **Shopping Cart and Products:** A Shopping Cart object contains multiple Product objects. The shopping cart has products, but products are not dependent on the shopping cart for their existence.

◆ **Composition:** where objects are composed of other objects, forming a “has-a” style of relationship. The objects that comprise the internal objects cannot exist independently of the containing object.

◆ **Aggregation:** where one object “has” another object as part of it, but the two objects can exist independently of each other.

Common mistake

Confusing composition and aggregation

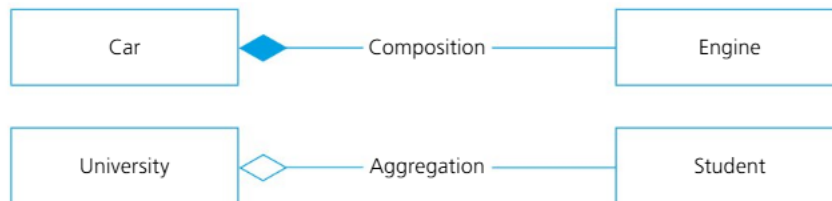
It is very easy and common to get confused as to which is which. The key difference is the lifecycle dependency between objects. Use composition when objects are part of a whole-of-lifecycle, and aggregation when the contained object can exist independently.

- **Computer System and Peripheral Devices:** A Computer System object can have references to Keyboard, Monitor, Mouse, and so on. The computer system has these peripheral devices, but these devices can be used with other computer systems.
- **Airline and Airplanes:** An Airline has a fleet of Airplane objects. The airline owns airplanes, but airplanes can be sold or transferred to other airlines and continue to operate.

Sometimes the distinction between composition and aggregation can seem arbitrary and open for debate. Consider, for instance, the example of Car and Engine. In some contexts, such as at a mechanic's garage, it could be argued the engine can be configured to function outside of the car for testing purposes. This subtlety is where your understanding of the context of the problem at hand is important: what might be correct for one question or scenario may be different for another. (Note that, even in the mechanic's shop, you could argue an Engine lacks autonomy or control over its own lifecycle; it is the containing object – the mechanic's garage – that has that role in that edge case.)

Relationships in UML

Composition and aggregation relationships can be depicted in UML diagrams using a diamond-pointed arrow that is filled for composition, and hollow for aggregation.



UML styling of composition and aggregation

Example

Consider the following scenario, where a Person object has an Address object as an instance variable.



UML Person has an Address (aggregation)

Is this an example of composition or aggregation? Take a moment to consider before reading on.

The appropriate guiding question is: Can each exist independently of the other? While ambiguity exists here and a case could be made either way, the situation would most likely favour that of aggregation. The Person object, as the owning class, doesn't have control over the lifecycle of the Address. If a Person moves house, or is deleted from the application, that Address may continue to exist. In this scenario, an Address may, in fact, be used by multiple Person objects at once, so one Person ceasing to use it wouldn't have much bearing on it at all.



ACTIVITY

Social skills and thinking skills: Brainstorm relationships

Draw UML diagrams based on various real-world scenarios using composition and aggregation, splitting the class into halves: one side being composition, the other aggregation. Each side has to explain their rationale for why the scenario is an example of their relationship. Whichever team comes up with the most scenarios wins.

B3.2.5 Design patterns in OOP



What are the implications of having, or not having, knowledge?

In OOP, knowledge of design patterns, best practices and anti-patterns significantly impacts the quality and maintainability of software.

Design patterns are common approaches to solving problems that are seen time and time again in software design. They are best practices formulated by experienced object-oriented software developers. Design patterns are useful because they provide tested, proven development paradigms, thereby improving code readability, reusability and reliability.

Some of the most commonly used design patterns that are relevant for beginners in object-oriented programming are the *singleton pattern*, the *factory pattern* and the *observer pattern*.

● Linking question

How can design patterns in OOP facilitate the architecture of scalable and maintainable machine learning models? (A4)

◆ **Singleton pattern:** a class that is designed only ever to have one instance instantiated throughout the lifecycle of the program.

◆ **Factory pattern:** a design pattern that provides an alternative interface for creating objects in contrast to normal constructor-based instantiation.

■ Singleton pattern

When using a **singleton pattern**, there will only ever be one instance of the class. That one instance will be made available globally throughout the project. It is often used for resource management, such as maintaining an open connection to a database or network location, or for settings management. In these scenarios, there is no need for more than one instance to do the job.

■ Factory pattern

The **factory pattern** is used for providing a *factory* that can be used for creating a range of similar objects that will adhere to a common interface.

A graphical user interface toolkit might use a factory method to create windows, buttons or other UI elements. For instance, a button factory can return buttons of different styles using a *Primary*, *Secondary* or *Alert* colour scheme, or perhaps completely different looks, such as classic vs modern.

◆ **Observer pattern:** provides a one-to-many link between objects to notify objects of changes in state via a subscription-style service.

■ Observer pattern

The **observer pattern** is used to maintain a list of dependent objects that have subscribed to it so that they receive notifications when an event occurs. The observer pattern allows the observing object to communicate changes to other objects that are interested in those changes. This pattern is widely used in implementing distributed event handling systems such as in web pages, where you might add event listeners to handle user input (clicks, keyboard events, and so on).

■ Interface

Before looking at how to apply each pattern, it is worth noting Java examples that demonstrate the use of an interface. The interface is not in your syllabus, but a quick introduction is merited as it is the right tool for the job to demonstrate these patterns.

An interface is a construct that defines a set of methods that implementing classes must provide, without specifying *how* these methods should be implemented. It allows different classes to interact with each other through a common set of behaviours, ensuring consistency and interoperability. Interfaces are similar to, but not quite the same as, abstract classes. They differ in the following ways:

- Interfaces do not store state (no instance variables). They simply define methods that need to be implemented.
- A class can implement multiple interfaces, whereas using normal inheritance is limited to only one parent class.
- Interfaces do not have constructors as they are not concerned with the logic of implementation (no variables, remember). They are just focused on providing a list of methods that require implementation.

An example might be an interface called `School`. This interface defines methods `attendClass()`, `doHomework()`, `takeExam()`. Different students might implement these methods differently, but they must implement all of them in one form or another.

Java

```
/* The interface designs a contract specifying methods that must be provided by
/* those classes that implement it. */
interface School {
    void attendClass();
    void doHomework();
    void takeExam();
}

class StudentA implements School {
    public void attendClass() {
        System.out.println("Student A attends class online via Zoom.");
    }
    public void doHomework() {
        System.out.println("Student A does homework in the early morning with a
        quiet background.");
    }
}
```

```

        public void takeExam() {
            System.out.println("Student A prefers taking exams in a quiet
classroom.");
        }
    }
    class StudentB implements School {
        public void attendClass() {
            System.out.println("Student B attends class in person at the school.");
        }
        public void doHomework() {
            System.out.println("Student B does homework late at night with music
playing.");
        }
        public void takeExam() {
            System.out.println("Student B takes exams online with open book
resources.");
        }
    }
    public class Main {
        public static void main(String[] args) {
            StudentA studentA = new StudentA();
            StudentB studentB = new StudentB();
            studentA.attendClass();
            studentA.doHomework();
            studentA.takeExam();
            studentB.attendClass();
            studentB.doHomework();
            studentB.takeExam();
        }
    }
}

```

■ Application of design patterns

Singleton example

The following code is an example of a singleton pattern that can be used to provide access to application settings that have been stored into a settings.json file. The example content of such a file follows:

```

{
    "databaseUrl": "postgresql://user:password@host:port/database",
    "timeout": 10
}

```

Note: Java users, if you plan to implement this, you will need to add the `org.json` library. Your IDE should allow you to easily add dependencies to other libraries.

Python

```
import json

class ConfigManager:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(ConfigManager, cls).__new__(cls)
            cls._instance.load_settings()
        return cls._instance
    def load_settings(self):
        try:
            with open("settings.json", "r") as f:
                self.settings = json.load(f)
        except FileNotFoundError:
            print("Error: The settings file was not found.")
            self.settings = {}
        except json.JSONDecodeError:
            print("Error: JSON decode error in settings file.")
            self.settings = {}
    def get_setting(self, key):
        return self.settings.get(key)

if __name__ == "__main__":
    config_manager = ConfigManager() # Initialize the singleton
    database_url = config_manager.get_setting("databaseUrl")
    timeout = config_manager.get_setting("timeout")
    print(f"Database URL: {database_url}")
    print(f"Timeout: {timeout}")
```

Java

```
import org.json.JSONObject;
import org.json.JSONTokener;
import java.io.FileInputStream;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class ConfigManager {
    private static ConfigManager instance;
    private Map<String, Object> settings;
    // Private constructor to prevent instantiation
    private ConfigManager() {
        loadSettings();
    }
    // Public method to get the instance
```

```

public static synchronized ConfigManager getInstance() {
    if (instance == null) {
        instance = new ConfigManager();
    }
    return instance;
}
// Load settings from file
private void loadSettings() {
    try (FileInputStream inputStream = new FileInputStream("settings.json")) {
        JSONTokener tokenener = new JSONTokener(inputStream);
        JSONObject jsonObject = new JSONObject(tokenener);
        settings = toMap(jsonObject);
    } catch (IOException e) {
        e.printStackTrace();
        settings = new HashMap<>(); // Fallback to an empty map
    }
}
// Helper method to convert JSONObject to Map
private Map<String, Object> toMap(JSONObject jsonObject) {
    Map<String, Object> map = new HashMap<>();
    jsonObject.keys().forEachRemaining(key -> map.put(key, jsonObject.get(key)));
    return map;
}
// Get a setting value by key
public Object getSetting(String key) {
    return settings.get(key);
}
}

public class Application {
    public static void main(String[] args) {
        ConfigManager configManager = ConfigManager.getInstance(); // Initialize
        // the Singleton
        String databaseUrl = (String) configManager.getSetting("databaseUrl");
        Integer timeout = (Integer) configManager.getSetting("timeout");
        System.out.println("Database URL: " + databaseUrl);
        System.out.println("Timeout: " + timeout);
    }
}

```


Factory example

The following code is an example of the factory pattern.

Python

```
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

class AnimalFactory:
    @staticmethod
    def get_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        return None

# Usage
if __name__ == "__main__":
    factory = AnimalFactory()
    dog = factory.get_animal("dog")
    cat = factory.get_animal("cat")
    print(dog.speak()) # Output: Woof!
    print(cat.speak()) # Output: Meow!
```

Java

```
// Factory pattern example - Produce dogs and cats
interface Animal {
    String speak();
}

class Dog implements Animal {
    public String speak() {
        return "Woof!";
    }
}

class Cat implements Animal {
    public String speak() {
        return "Meow!";
    }
}
```

```

class AnimalFactory {
    public static Animal getAnimal(String animalType) {
        if ("dog".equalsIgnoreCase(animalType)) {
            return new Dog();
        } else if ("cat".equalsIgnoreCase(animalType)) {
            return new Cat();
        }
        return null; // or throw an exception
    }
}

public class Main {
    public static void main(String[] args) {
        Animal dog = AnimalFactory.getAnimal("dog");
        Animal cat = AnimalFactory.getAnimal("cat");
        System.out.println(dog.speak()); // Output: Woof!
        System.out.println(cat.speak()); // Output: Meow!
    }
}

```

Observer example

Finally, here is an example of the observer pattern at work:

Python

```

class NotificationService:
    def __init__(self):
        self._observers = []
    def attach(self, observer):
        self._observers.append(observer)
    def detach(self, observer):
        self._observers.remove(observer)
    def notify(self, message):
        for observer in self._observers:
            observer.update(message)

class Observer:
    def update(self, message):
        print(f"Received: {message}")

# Usage
notifier = NotificationService()
observer_a = Observer()
observer_b = Observer()
notifier.attach(observer_a)
notifier.attach(observer_b)
notifier.notify("Hello World!") # Output: Received: Hello World! from both

```

Java

```
// Observer pattern example - notification service
interface Observer {
    void update(String message);
}

class NotificationService {
    private List<Observer> observers = new ArrayList<>();
    public void attach(Observer observer) {
        observers.add(observer);
    }
    public void detach(Observer observer) {
        observers.remove(observer);
    }
    public void notifyObservers(String message) {
        for (Observer observer : observers) {
            observer.update(message);
        }
    }
}

class ConcreteObserver implements Observer {
    public void update(String message) {
        System.out.println("Received: " + message);
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        NotificationService notifier = new NotificationService();
        Observer observerA = new ConcreteObserver();
        Observer observerB = new ConcreteObserver();
        notifier.attach(observerA);
        notifier.attach(observerB);
        notifier.notifyObservers("Hello World!"); // Output: Received: Hello
                                                    // World! from both
    }
}
```

REVIEW QUESTIONS

- 1 In UML diagrams, which symbol is used to represent inheritance between two classes?
 - a A dashed line with an arrow
 - b A solid line with a hollow arrow
 - c A solid line with a filled arrow
 - d A dashed line without any arrows

- 2 What keyword is used in Java to inherit a class?
 - a Implements
 - b Extends
 - c Inherits
 - d Superclass
- 3 Which principle of OOP is primarily used to enhance code flexibility and maintainability through interfaces?
 - a Encapsulation
 - b Inheritance
 - c Polymorphism
 - d Abstraction
- 4 What is required for method overriding to occur in object-oriented programming?
 - a The method must have the same name and different parameters in the subclass
 - b The method must have the same name and parameter list in the subclass, and be marked with final
 - c The method must have the same name and parameter list in the subclass
 - d The method must have a different name but the same parameters in the subclass
- 5 Which statement is true about abstract classes?
 - a Abstract classes can be instantiated
 - b Abstract classes cannot have any method implementations
 - c Abstract classes can contain both abstract and implemented methods
 - d All methods in an abstract class must be abstract
- 6 What is the purpose of declaring a class as abstract?
 - a To force a class to provide implementations of all its methods
 - b To prevent the class from being instantiated directly
 - c To ensure that the class can only contain static methods
 - d To make the class available only to other classes in the same package
- 7 Which of the following best describes aggregation?
 - a A strong "has-a" relationship where the lifetime of the contained objects depends on the lifetime of the container
 - b A weak "has-a" relationship where the contained objects can exist independently of the container
 - c An "is-a" relationship between two entities
 - d None of the above
- 8 Which scenario is an example of composition?
 - a A library owns books
 - b A university has students
 - c An apartment building includes apartments
 - d A shopping cart contains products
- 9 Which design pattern ensures that a class has only one instance and provides a global point of access to it?
 - a Factory pattern
 - b Singleton pattern
 - c Observer pattern
 - d Builder pattern

- 10 In the observer design pattern, what is the role of the “Subject”?
 - a To notify all observers about any changes
 - b To keep track of all dependencies
 - c To update the state of various subjects
 - d To request updates from observers
- 11 Describe why inheritance is considered a powerful feature of OOP.
- 12 Discuss how polymorphism enhances software maintainability.
- 13 Discuss two benefits that abstract classes provide over using normal inheritance.
- 14 Outline an example, not based on any given in this book, that illustrates the difference between composition and aggregation.
- 15 Describe the factory design pattern and give an example of its use.

PROGRAMMING EXERCISES

Some exercises require you to download files for processing or performing calculations. Those files can be downloaded from: <https://github.com/paulbaumgarten/hodder-ibdp-computerscience>

1 Extend bank accounts

Implement the bank account example provided at the end of Section B3.1, and debug any transcription errors to ensure it behaves as expected before continuing.

- ☐ Download the following files from the B3 folder in the Github repository:
 - names.txt
 - bank-transactions.txt
- ☐ Use the file reading techniques from B2.5 to read each file into an array of strings (one string per line of the file).
- ☐ Create an array of bank accounts, one for each person in your array, from the names.txt file.
- ☐ Process the list of transactions in bank-transactions.txt.
- ☐ Apply interest calculations on all accounts.
- ☐ Print all accounts' final balances.

Do you get the correct final balances for your account holders, as shown below?

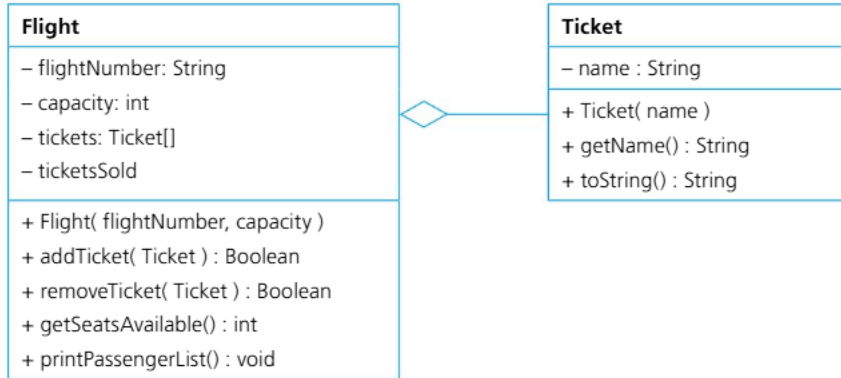
Account 1001: Eustolia has balance \$5571.3
 Account 1002: Nathan has balance \$9515.1
 Account 1003: Milissa has balance \$61.95
 Account 1004: Willie has balance \$1912.05
 Account 1005: Hoyt has balance \$4697.7
 Account 1006: Alexandria has balance \$2461.2
 Account 1007: Clelia has balance \$3311.7
 Account 1008: Alpha has balance \$1942.5
 Account 1009: Delbert has balance \$4670.4
 Account 1010: Boyd has balance \$547.05
 Account 1011: Milton has balance \$2331.0
 Account 1012: Vivan has balance \$1275.75
 Account 1013: Constance has balance \$2983.05

Account 1014: Hilma has balance \$258.3
 Account 1015: Irving has balance \$76.65
 Account 1016: Carie has balance \$2654.4
 Account 1017: Nicky has balance \$297.15
 Account 1018: Adele has balance \$3287.55
 Account 1019: Carlene has balance \$882.0
 Account 1020: Hermina has balance \$2.1
 Account 1021: Ayana has balance \$586.95
 Account 1022: Frederica has balance \$261.45
 Account 1023: Arianna has balance \$541.8
 Account 1024: Zandra has balance \$725.55
 Account 1025: Vina has balance \$2553.6

2 Flight reservation system

You are constructing a ticket reservation system for a budget airline. This airline doesn't have seating classes (no first class or business class – everyone sits in economy), and doesn't accept seat reservations. The only thing the airline is interested in is ensuring enough seats are available on each flight for the tickets it sells.

You are taking over from another programmer who started designing your classes for you. The following is the UML they created.



■ UML Flight and Ticket

Create the Ticket class and Flight class in code, to adhere to the following rules:

- ☐ The `capacity` variable in the Flight constructor indicates the number of seats available on a given flight. Use this variable to determine the size of your tickets array.
- ☐ Implement `addTicket()` and `removeTicket()` to add or remove a ticket from the tickets array. `ticketsSold` should increment whenever a new ticket is added, and decrement whenever a ticket is removed. Refer to the `CollectionOfThings` example in Section B3.1.3 for hints on how to implement this.
- ☐ `getSeatsAvailable()` should just be the result of `capacity` minus `ticketsSold`.
- ☐ `printPassengerList()` should print a list of all the names of tickets for a given flight.

The following code is example main code for testing the result, and an indication of what the output should resemble.

Python

```

if __name__ == "__main__":
    cx619 = Flight("CX 619", 280) # Cathay - HKG-SIN
    aa6914 = Flight("AA 6914", 266) # American - JFK-LHR
    ek89 = Flight("EK 89", 354) # Emirates - DBX-GVA
    jordan = Ticket("Jordan Deckard")
    cx619.addTicket(Ticket("Taylor Ripley"))
    cx619.addTicket(jordan)
    cx619.addTicket(Ticket("Casey Neo"))
    aa6914.addTicket(Ticket("Cameron Quaid"))
    aa6914.addTicket(Ticket("Phoenix Andor"))
    cx619.printPassengerList()
    print(cx619.getSeatsAvailable())
    cx619.removeTicket(jordan)
    cx619.printPassengerList()
  
```

Java

```

class Main {
    public static void main(String[] args) {
        Flight cx619 = new Flight("CX 619", 280); // Cathay - HKG-SIN
        Flight aa6914 = new Flight("AA 6914", 266);
        // American - JFK-LHR
        Flight ek89 = new Flight("EK 89", 354); // Emirates - DBX-GVA
        Ticket jordan = Ticket("Jordan Deckard");
        cx619.addTicket(new Ticket("Taylor Ripley"));
        cx619.addTicket(jordan);
        cx619.addTicket(new Ticket("Casey Neo"));
        aa6914.addTicket(new Ticket("Cameron Quaid"));
        aa6914.addTicket(new Ticket("Phoenix Andor"));
        cx619.printPassengerList();
        System.out.println(cx619.getSeatsAvailable());
        cx619.removeTicket(jordan);
        cx619.printPassengerList();
        System.out.println(cx619.getSeatsAvailable());
    }
}

```

Anticipated output:

Flight CX 619 has passengers:

Taylor Ripley

Jordan Deckard

Casey Neo

277

Flight CX 619 has passengers:

Taylor Ripley

Casey Neo

278

3 School enrolments and grade book

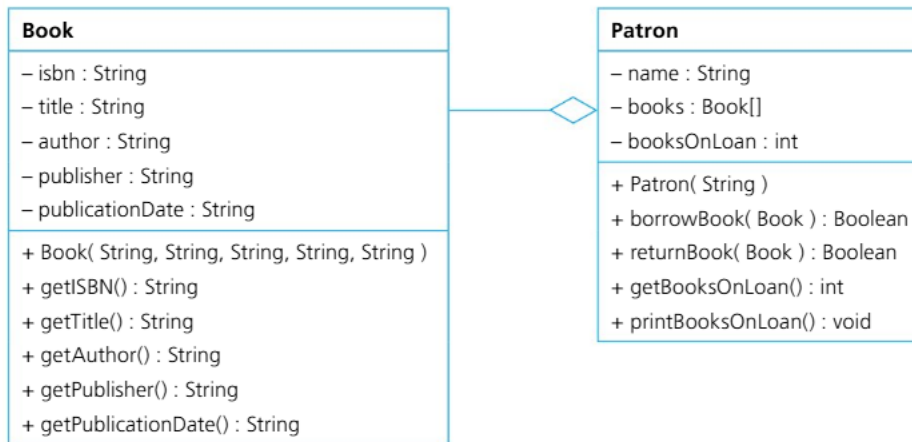
Implement the students and grades example as provided in the end of Section B3.1, and debug any transcription errors to ensure it behaves as expected before continuing.

Extend the scenario to include a new class called "Course" that contains an array of students who are enrolled in it.

- ☐ Course should contain an `addStudent()` and `removeStudent()` method.
- ☐ Course should have the following additional functions:
 - `printClassList()` generates a list of all student names (in name sort order)
 - `getClassAverage()` returns the average of all student scores.

4 Library system

Create a system to manage book loans for a library.



■ UML Book and Patron

- ☐ Create a Book class and Patron class, as per the UML diagram.
- ☐ Download the following files from the B3 folder in the Github repository:
 - names.txt
 - books.txt or books.csv
 - library-transactions.txt
- ☐ Use the file reading techniques from Section B2.5 to read each file into an array of strings.
- ☐ Create an array of patrons, one for each name in your array from the names.txt file, and create an array of books based on the data in books.txt.
- ☐ Process the list of transactions in library-transactions.txt, subject to the following:
 - Each patron can only have a maximum of three books on loan at a time.
 - If a patron attempts to borrow a fourth book, it should be denied.
 - A book can be borrowed more than once at a time (imagine there are unlimited copies of each book).

At the end, print a summary of each patron's current books. The final list starts with:

Person Eustolia has these books:

- 9780141030142 Memory Keeper's Daughter,The
- 9780099387916 Birdsong
- 9780006498407 Angela's Ashes:A Memoir of a Childhood

Person Nathan has these books:

- 9780099419785 To Kill a Mockingbird
- 9781904994497 Guinness World Records 2010
- 9780140237504 Catcher in the Rye,The

Person Milissa has these books:

- 9780701181840 Nigella Express
- 9780099450252 Curious Incident of the Dog in the Night-time,The

Person Willie has these books:

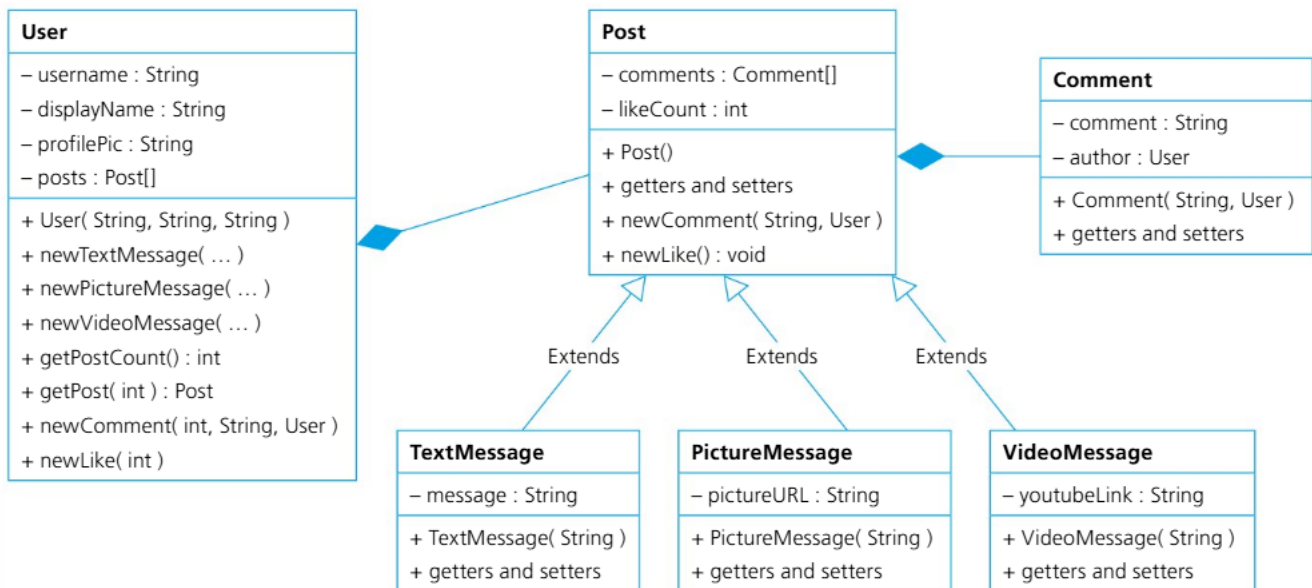
- 9780563384304 Delia's How to Cook:(Bk.1)
- 9780590112895 Subtle Knife,The:His Dark Materials
- 9780747581109 Harry Potter and the Half-Blood Prince

How many times did someone attempt to borrow a book over their limit? You should get 1125.

How many books are on loan at the end of the sequence? You should get 51.

5 Social-media platform (HL)

Use the following UML as the basis for creating an OOP application for a social-media platform.



■ UML social-media platform

For a Java implementation, you may assume the maximum number of posts per user is ten, and the maximum number of comments per post is ten.

Your final program should be able to:

- ☐ create a new user
 - ☐ allow a user to create a post, via their user object's `.newTextMessage()`, `.newPictureMessage()` or `.newVideoMessage()` function
 - ☐ allow a comment to be added to any post via the user object's `.newComment()` function, where the integer value represents the index of the relevant item in the posts array, and the `String, User` parameters are then passed to the `posts[i].newComment(String, User)` function.
- The user object's `.newLike()` works similarly, using the integer as the index of the post, to then call the `.newLike()` on the relevant item in the posts array.

The following code is test code for your main function:

Python

```

import random
if __name__ == "__main__":
    frodo = User("frodo", "Frodo Baggins", "frodo.png")
    gandalf = User("gandalf", "Gandalf the Grey", "gandalf.png")
    samwise = User("samwise", "Samwise Gamgee", "samwise.png")
    aragorn = User("aragorn", "Aragorn", "aragorn.png")
    # Create posts
    frodo.new_text_message("Just finished my quest to destroy the One Ring.
    What an adventure! #MissionAccomplished #RingBearer")
    frodo.new_picture_message("Pic of me after the quest! #MountDoom #Mordor")
    aragorn.new_text_message("Reunited with my love, Arwen. Forever grateful
    for her love and support. ❤️ #Elessar #LoveAndDestiny")
  
```



```

gandalf.new_video_message("Just having some fun with my staff and
showing off a little wizardry on the dance floor! 🧙🏻 #GandalfTheDancer
#WizardGrooves")
# Create comments
frodo.new_comment(0, "Mr. Frodo! I'm so proud of you! You're the bravest
hobbit I know. #TrueFriendship #HobbitHeroes", samwise)
frodo.new_comment(0, "Frodo, you have my gratitude and respect. Your
sacrifice has saved Middle-earth. #KingOfGondor #HeroicDeeds", aragorn)
frodo.new_comment(0, "Well done, Frodo! You've shown incredible strength
and determination. The world is safer because of you. #RingDestroyer
#WizardPride", gandalf)
frodo.new_comment(1, "Who's that great looking bloke next to you?",
samwise)
frodo.new_comment(1, "Amazing!", aragorn)
aragorn.new_comment(0, "Aragorn, may your love with Arwen be as enduring
as the light of the Silmarils. #LoveAndHope #FellowshipForever", frodo)
aragorn.new_comment(0, "Aragorn, you have found true love. Cherish it
always. #Shieldmaiden #HappilyEverAfter", gandalf)
gandalf.new_comment(0, "Love it! #WizardsCanDance", frodo)
gandalf.new_comment(0, "rofl", samwise)
# Create likes
frodo.new_like(1)
frodo.new_like(1)
aragorn.new_like(0)
r = random.randint(1001, 2000)
for i in range(r):
    gandalf.new_like(0)

```

Java

```

import java.util.Random;
class Main {
    public static void main(String[] args) {
        User frodo = new User("frodo", "Frodo Baggins", "frodo.png");
        User gandalf = new User("gandalf", "Gandalf the Grey", "gandalf.png");
        User samwise = new User("samwise", "Samwise Gamgee", "samwise.png");
        User aragorn = new User("aragorn", "Aragorn", "aragorn.png");
        // Create posts
        frodo.newTextMessage("Just finished my quest to destroy the One Ring.
What an adventure! #MissionAccomplished #RingBearer");
        frodo.newPictureMessage("Pic of me after the quest! #MountDoom
#Mordor");
        aragorn.newTextMessage("Reunited with my love, Arwen. Forever grateful
for her love and support. ❤️ #Elessar #LoveAndDestiny");
        gandalf.newVideoMessage("Just having some fun with my staff
and showing off a little wizardry on the dance floor! 🧙🏻
#GandalfTheDancer #WizardGrooves");
    }
}

```

```

// Create comments
frodo.newComment(0, "Mr. Frodo! I'm so proud of you! You're the
bravest hobbit I know. #TrueFriendship #HobbitHeroes", samwise);
frodo.newComment(0, "Frodo, you have my gratitude and respect. Your
sacrifice has saved Middle-earth. #KingOfGondor #HeroicDeeds", aragorn);
frodo.newComment(0, "Well done, Frodo! You've shown incredible
strength and determination. The world is safer because of you.
#RingDestroyer #WizardPride", gandalf);
frodo.newComment(1, "Who's that great looking bloke next to you?",
samwise);
frodo.newComment(1, "Amazing!", aragorn);
aragorn.newComment(0, "Aragorn, may your love with Arwen
be as enduring as the light of the Silmarils. #LoveAndHope
#FellowshipForever", frodo);
aragorn.newComment(0, "Aragorn, you have found true love. Cherish it
always. #Shieldmaiden #HappilyEverAfter", gandalf);
gandalf.newComment(0, "Love it! #WizardsCanDance", frodo);
gandalf.newComment(0, "rofl", samwise);
// Create likes
frodo.newLike(1);
frodo.newLike(1);
aragorn.newLike(0);
Random random = new Random();
int randomNumber = random.nextInt(1001) + 1000;
for (int i=0; i<randomNumber; i++) {
    gandalf.newLike(0);
}
}
}

```

EXAM PRACTICE QUESTIONS

1 Customer loyalty system

A chain of stores has launched a new customer loyalty program, where each dollar customers spend accrues loyalty points that can be exchanged for discounts on future purchases. An object-oriented program has been created to manage the loyalty program.

The following classes exist in the system:

- ☐ Customer: Represents customer information, including a list of all their purchase history items
- ☐ Transaction: Represents the purchase of a single item by a customer.

The UML diagram for the Customer class is provided below:

Customer

- int	id
- String	name
- long	balance
- Transaction[]	history
- long	historyItemCount
+ Customer (int id, String name)	addTransaction(Transaction item)
+ void	spendPoints(String description, int points)
+ Boolean	getTransactionByID(int id)
+ Transaction	getTransactionByDesc (String description)
+ Transaction	getBalance()
+ long	getName()
+ String	

(Up to 10,000 items)

UML Customer class

- State the relationship between Customer and Transaction. [1]
- Construct a simplified UML diagram showing the relationships between Customer and Transaction. [2]
- Outline the significance of the minus sign in front of long balance in the UML diagram. [2]
- Construct the code for the constructor of Customer. [3]

A Transaction object has two properties: a string containing a description of the item purchased and a long integer containing the cost / value of the item in dollars (you can assume cents are not used). The following code forms the basis of the Transaction class:

Python

```
class Transaction:
    def __init__(self, description, points):
        self.description = description
        self.points = points
    # Getters and setters for description and points
```

Java

```
public class Transaction {
    private String description;
    private int points;
    public Transaction(String description; int points) {
        this.description = description;
        this.points = points;
    }
    // Getters and setters for description and points
}
```

Additionally, the following code describes the functionality of the addItem(Transaction item) function in the Customer class:

Python

```
def add_item(item):
    history[ self.history_item_count ] = item
    self.history_item_count += 1
    self.balance += item.get_points()
```

Java

```
public void addItem(Transaction item) {
    history[ historyItemCount++ ] = item;
    balance += item.getPoints();
}
```

- e Describe the purpose of the historyItemCount property. [2]
- f Construct the code for the getTransactionByDesc(String description) function using a linear search. [4]
- g To improve the efficiency of searching for transactions within each customer object, it has been decided to create a sort() function within Customer that will sort the transactions alphabetically by description. This algorithm will use a selection sort for the task. Construct the code for the new sort() function to be added to the Customer class. [6]
- h Now that the history array is sorted, construct new code for getTransactionByDesc(String description) that will implement a binary search algorithm. [5]
- i The spendPoints() function should first check whether the customer has enough points for the transaction and return false if not. Assuming enough points exist, it should create a new Transaction object that is added to the history, and deduct the points spent from the balance. Construct the code for the spendPoints() function. [6]
- j The getBalance() function should iterate over all items in the history to calculate the correct balance, then update the value stored in the balance property appropriately, and finally return that value. Construct the code for the getBalance() function. [6]
- k The main() of the program contains the following test code. State the output from this block of code. [4]

Python

```
if __name__ == "__main__":
    t = new Transaction(250, "Special deal")
    # Create list of 10 None items to replicate an empty array
    customers = [ None for _ in range(10) ]
    customers[0] = Customer(0, "Ava")
    customers[1] = Customer(1, "Brian")
    customers[2] = Customer(2, "Cherry")
    customers[0].add_item(50, "Burger meal deal")
    customers[0].add_item(Transaction(100, "Birthday bonus"))
    customers[1].add_item(200, "Bluetooth earphones")
    customers[1].spend_points(50, "Discount for shopping")
    customers[2].spend_points(80, "Discount for shopping")
    customers[2].add_item(t)
    print( customers[0].get_balance() )
    print( customers[1].get_balance() )
    print( customers[2].get_balance() )
    print( customers[3].get_balance() )
```


Java

```

public static void main(String[] args) {
    Transaction[] t = new Transaction(250, "Special deal");
    Customer[] customers = new Customer[10];
    customers[0] = new Customer(0, "Ava");
    customers[1] = new Customer(1, "Brian");
    customers[2] = new Customer(2, "Cherry");
    customers[0].addItem(50, "Burger meal deal");
    customers[0].addItem(new Transaction(100, "Birthday bonus"));
    customers[1].addItem(200, "Bluetooth earphones");
    customers[1].spendPoints(50, "Discount for shopping");
    customers[2].spendPoints(80, "Discount for shopping");
    customers[2].addItem(t);
    System.out.println(customers[0].getBalance());
    System.out.println(customers[1].getBalance());
    System.out.println(customers[2].getBalance());
    System.out.println(customers[3].getBalance());
}

```

2 Animal shelter

An animal rescue shelter requires a computer system to manage the animals under its care and the adoption process.

When an animal is brought to the shelter, it is given an ID, and its species, age, health status and other relevant details are recorded in the system. When an animal is adopted, its record is updated to reflect the change in status.

Animals are identified by a unique ID, which is a combination of letters and numbers (e.g. C4T00123). This is used to track their information in the system.

A programmer created the classes AnimalShelter and Animal to model the situation above.

Python

```

class AnimalShelter:
    def __init__(self, location, capacity):
        self.location = location
        self.animals = [ None for _ in range(capacity) ]
    def get_location(self):
        return self.location
    def get_capacity(self):
        return len(animals)
    def find_animal(self, id:str):
        # Method to find an animal by an ID in the list and return the index
class Animal:
    def __init__(self, id, species):
        # Missing code to initialize _id, _species, _age, _is_healthy
    def set_age(self, age):
        self._age = age
    def set_health_status(self, is_healthy):
        self._is_healthy = is_healthy

```



```

def get_id(self):
    return self._id
def get_species(self):
    return self._species
def get_health_status(self):
    return self._is_healthy
def get_age(self):
    return self._age

```

Java

```

public class AnimalShelter {
    private Animal animals[];
    private String location;
    AnimalShelter(String location, int capacity) {
        this.location = location;
        this.animals = new Animal[capacity];
    }
    String getLocation() {
        return location;
    }
    public int getCapacity() {
        return animals.length;
    }
    public int findAnimal(String id) {
        // Method to find an animal by ID in the array and return the index
    }
}

public class Animal {
    private String id;
    private String species;
    private int age;
    private boolean isHealthy;
    public Animal(String id, String species) {
        // Missing code
    }
    public void setAge(int age) {
        this.age = age;
    }
    public void setHealthStatus(boolean isHealthy) {
        this.isHealthy = isHealthy;
    }
    public String getId() {
        return id;
    }
}

```

```

    public String getSpecies() {
        return species;
    }
    public boolean getHealthStatus() {
        return isHealthy;
    }
    public int getAge() {
        return age;
    }
}

```

- a Explain the purpose of encapsulation in object-oriented programming and how it is applied in the classes above. [2]
- b Construct the missing code for the constructor of the Animal class. [3]
- c Explain the use of the keyword `this` in the `setAge` method of the Animal class. [2]
- d Construct code to create an instance of the Animal class with the ID "C4T00123" and species "Cat". [2]
- e Construct code to set the age of the object created above to 3 years. [2]
- f Construct the method `addAnimal(Animal a)` that will add an Animal to the first empty position of the array `animals[]` in the `AnimalShelter` class and return the position at which it has added the animal. If the array is full and the animal cannot be added, the method should return `-1`. [5]

HL extension

Two subclasses, Dog and Cat, are created.

Python

```

class Dog(Animal):
    def __init__(self, id, is_vaccinated):
        super().__init__(id, "Dog")
        self._is_vaccinated = is_vaccinated
    def get_vaccination_status(self):
        return self._is_vaccinated
class Cat(Animal):
    def __init__(self, id, is_neutered):
        super().__init__(id, "Cat")
        self._is_neutered = is_neutered
    def get_neutered_status(self):
        return self._is_neutered

```

Java

```

public class Dog extends Animal {
    private boolean isVaccinated;
    public Dog(String id, boolean isVaccinated) {
        super(id, "Dog");
        this.isVaccinated = isVaccinated;
    }
}

```

```

        public boolean getVaccinationStatus() {
            return isVaccinated;
        }
    }
    public class Cat extends Animal {
        private boolean isNeutered;
        public Cat(String id, boolean isNeutered) {
            super(id, "Cat");
            this.isNeutered = isNeutered;
        }
        public boolean getNeuteredStatus() {
            return isNeutered;
        }
    }
}

```

g Construct a UML diagram that shows the relationships between the AnimalShelter, Animal, Dog and Cat classes. Include only the class names and relationships. [3]

The Animal class needs a method that returns a description of the animal, including its ID and species.

h Construct a method getDescription() in the Animal class that returns a string describing the animal. [2]

The array animals[] in the AnimalShelter class is used to store instances of any kind of animal, including Dog and Cat.

i Justify why Animal is a suitable type for this array. [2]

The shelter has a program that recognizes the loyalty of volunteers by rewarding every tenth time someone donates time to work at the shelter with a free pet-food voucher. The method to print this voucher has been implemented in the static method Rewards.printPetFoodVoucher().

(Note for clarity that the system is not keeping track of how many times each individual person volunteers; just the raw count for every tenth person who shows up – so one individual may get the voucher the first time they volunteer, or may attend for weeks without receiving a voucher!)

A getVolunteerCount() method has been added to the AnimalShelter class, which returns the current count of volunteers.

j Describe the necessary changes to the AnimalShelter class and any other methods to integrate the volunteer reward program into the system. [5]

The method removeAnimal() in the AnimalShelter class searches the array for an Animal object with a specified ID and removes it by setting that index to null. The method returns a reference to the Animal object that has been removed. You may assume that an Animal with the ID exists in the array.

k Construct the removeAnimal(String ID) method. [5]

3 Streamify music service

An online music and multimedia platform, Streamify, provides users with access to millions of tracks, podcasts and videos. To manage the vast collection, Streamify uses a computer system to keep track of the digital media, user preferences and playlists.

Each track is identified by a unique identifier, and contains metadata including the title, artist, duration and genre. Users can create their own playlists by adding tracks to a personalized list.

A programmer created the classes MediaLibrary and Track to model this situation.

Python

```

class MediaLibrary:
    def __init__(self, name, capacity):
        self._name = name
        self._tracks = []

```

```

def get_name(self):
    return self._name
def get_size(self):
    return len(self._tracks)
def find_track(self, id):
    # Method to locate a track in the list by its ID
    # Not yet implemented

class Track:
    def __init__(self, id, title, artist, duration, genre):
        self._id = id
        self._title = title
        self._artist = artist
        self._duration = duration
        self._genre = genre
    # Accessor methods for each attribute

```

Java

```

public class MediaLibrary {
    private ArrayList<Track> tracks;
    private String name;
    MediaLibrary(String name) {
        this.name = name;
        this.tracks = new ArrayList<>();
    }
    String getName() {
        return name;
    }
    public int getSize() {
        return tracks.size();
    }
    public int findTrack(String id) {
        // Method to locate a track in the array by its ID
        // Not yet implemented
    }
}

public class Track {
    private String id;
    private String title;
    private String artist;
    private int duration; // in seconds
    private String genre;

```

```

public Track(String id, String title, String artist, int duration, String
genre) {
    this.id = id;
    this.title = title;
    this.artist = artist;
    this.duration = duration;
    this.genre = genre;
}
// Accessor methods for each attribute
}

```

- a Outline why the programmer may have decided to use a list instead of an array to store the tracks within the MediaLibrary class. [2]
 - b Describe the relationship between the classes Track and MediaLibrary. [3]
 - c Discuss the importance of using the keyword `this` in the constructor of the Track class. [2]
 - d Construct code to create an instance of the Track class with the following details:
 - ID: TRK12345
 - Title: Oceans
 - Artist: Dive Deep
 - Duration: 215 seconds
 - Genre: Ambient
 [2]
 - e Construct a method in the MediaLibrary class that allows a user to add a Track to the library. [2]
- Streamify allows users to search for media based on genre or artist.
- f Construct a method `searchByGenre(String genre)` in the MediaLibrary class that returns an array of Track objects that match the genre. [4]
 - g Discuss the process of overriding methods of the Track class if it were to be extended by Podcast and Video classes. [4]
- Two classes, Playlist and User, are now introduced.

Python

```

class Playlist:
    def __init__(self, name):
        self.name = name
        self.track_list = []
    # Methods to add and remove tracks from the playlist
class User:
    def __init__(self, username):
        self.username = username
        self.playlists = []
    # Methods to create and manage playlists

```


Java

```

public class Playlist {
    private String name;
    private ArrayList<Track> trackList;
    public Playlist(String name) {
        this.name = name;
        this.trackList = new ArrayList<Track>();
    }
    // Methods to add and remove tracks from the playlist
}

public class User {
    private String username;
    private ArrayList<Playlist> playlists;
    public User(String username) {
        this.username = username;
        this.playlists = new ArrayList<Playlist>();
    }
    // Methods to create and manage playlists
}

```

- h Construct a UML diagram that shows the relationships between the MediaLibrary, Track, Playlist and User classes. Include only the relationships without the attributes or methods of each class. [4]
- Streamify calculates the total duration of a playlist by summing the duration of each track in the playlist.
- i Construct a method in the Playlist class that returns the total duration of the playlist. [2]
- j Explain why the Playlist class uses a list to store tracks instead of an array. [2]
- The User class can have multiple playlists, and the Playlist class contains multiple tracks.
- k Discuss why composition is used between the User and Playlist classes, and between the Playlist and Track classes. [2]
- To enhance user engagement, Streamify introduces a feature that rewards users with a free month of premium subscription for every 100 tracks they add to their playlists.
- The method `rewardUser` has been added to the User class, which checks the total number of tracks across all playlists and rewards the user if they meet the criteria.
- l Without writing code, describe any changes required to the `addTrack` method in the Playlist class and the User class to make the new reward system work. [5]
- The `removeTrack` method of the Playlist class allows users to remove a track from their playlist by specifying the track's unique ID.
- m Construct the `removeTrack()` method for the Playlist class. [4]